

01 — Field Definitions & Phone Data Quality Audit

Audience: SF Admin, Data Team **Prerequisites:** Access to ameyo.my.salesforce.com (admin or at minimum "Customize Application" + "Modify All Data" on Account, Case)

Part A — Schema Additions

A.1 — Account.Support_Tier__c (Formula Field)

Purpose: Single derived field that resolves the customer's support tier from existing commercial signals. Used by IVR HTTP callout and Case Flow.

Attribute	Value
API Name	Support_Tier__c
Label	Support Tier
Object	Account
Type	Formula (Text)
Length	50
Help Text	Derived support tier based on commercial signals. Used for routing support calls and tickets.

Formula:

```
IF(
  AND(Hypercare_active__c = "Active", OR(ISBLANK(Hypercare_End_Date__c),
Hypercare_End_Date__c >= NOW()))),
  "HYPERCARE",
  IF(
    OR(Top_Customer__c = TRUE, Is_SAAS_Premium_Account__c = TRUE),
    "STRATEGIC",
    IF(
      AND(SAAS_Enterprise_Account__c = TRUE, OR(TEXT(RAG__c) = "GREEN", TEXT(RAG__c) =
"AMBER"))),
      "PREMIUM",
      "STANDARD"
    )
  )
)
```

Note: `Hypercare_active__c` is a picklist per `CLAUDE.md` — confirm actual picklist values in sandbox before deploying. Replace `"Active"` above with the true picklist value.

Page Layout: Add to Account Highlights panel. Add to all Account page layouts used by Support and CSM.

A.2 — Account.Support_Tier_Badge__c (Formula, for UI)

Emoji/text badge version for display.

Attribute	Value
API Name	Support_Tier_Badge__c
Label	Support Tier Badge
Object	Account
Type	Formula (Text)
Length	50

Formula:

```
CASE(Support_Tier__c,  
  "HYPERCARE", "🔴 HYPERCARE",  
  "STRATEGIC", "🟡 STRATEGIC",  
  "PREMIUM", "🔵 PREMIUM",  
  "STANDARD"  
)
```

Note: Salesforce formula fields support emoji via Unicode but display may vary by browser / font. If emojis render inconsistently, replace with text-only ("!! HYPERCARE", "** STRATEGIC", etc.).

A.3 — Case.Account_Revenue_Tier__c (Picklist)

Purpose: Stamped at Case creation. Immutable for the life of the Case (important — if Account changes tier later, the Case retains its creation-time tier for SLA tracking).

Attribute	Value
API Name	Account_Revenue_Tier__c
Label	Account Revenue Tier
Object	Case
Type	Picklist
Values	HYPERCARE, STRATEGIC, PREMIUM, STANDARD, UNKNOWN
Default	UNKNOWN
Required	No (set by Flow)
Help Text	Customer tier at time of Case creation. Stamped automatically; do not edit manually.

A.4 — Case.Account_RAG_at_Creation__c (Text)

Purpose: Snapshot of Account.RAG__c at Case creation. RAG changes over time; we need the creation-time value for SLA and analytics.

Attribute	Value
API Name	Account_RAG_at_Creation__c
Label	Account RAG at Creation
Object	Case
Type	Text
Length	20

A.5 — Case.Account_CSM_Name__c (Text)

Purpose: CSM name stamped on Case so L1 agent sees it without a lookup.

Attribute	Value
API Name	Account_CSM_Name__c
Label	Account CSM
Object	Case
Type	Text
Length	120

A.6 — Case.Is_Top_Customer__c (Checkbox)

Purpose: Mirror of `Account.Top_Customer__c` on Case for use in list views and assignment rules.

Attribute	Value
API Name	Is_Top_Customer__c
Label	Top Customer
Object	Case
Type	Checkbox
Default	Unchecked

A.7 — Case.Account_LTM_Revenue__c (Currency) — OPTIONAL

Purpose: Sum of last 12 months revenue from `accounts_revenue__c`, rolled up to the Case. Useful for triage context.

Attribute	Value
API Name	Account_LTM_Revenue__c
Label	Account LTM Revenue

Object	Case
Type	Currency
Decimal Places	2
Help Text	Sum of Revenue_Booked_Amount__c on accounts_revenue__c for the last 12 months. Stamped at Case creation.

Tradeoff: Adding this requires a second Get Records in the Flow (querying `accounts_revenue__c`). At ~100 Cases/day this is safe. If Case volume grows 10x, reconsider whether to compute this at ticket creation or use a nightly rollup on Account.

A.8 — Account.LTM_Revenue__c (Currency, Rollup or Nightly Batch) — OPTIONAL

Alternative to A.7. Compute LTM revenue on the Account once (nightly batch or roll-up summary), then Flow just reads it.

Preferred if Case volume is high OR if `Account_LTM_Revenue__c` is wanted in multiple places beyond Case.

Part B — Phone Number Data Quality Audit

The single biggest risk to the call-channel leg. If Account.Phone or Contact.Phone is inconsistent or blank, ANI → Account lookup fails and calls fall to the Standard queue regardless of actual tier.

B.1 — Baseline: How Many Accounts Have Phone Numbers?

Run this first to understand the scope of the problem.

```
SELECT
  COUNT(Id)                TotalAccounts,
  COUNT(Phone)             WithPhone,
  COUNT(Id) - COUNT(Phone) WithoutPhone
FROM Account
WHERE IsDeleted = FALSE
```

Expected output shape: `TotalAccounts | WithPhone | WithoutPhone` **Interpretation:** If `WithPhone / TotalAccounts < 60%`, phone cleanup is a major project on its own.

B.2 — Phone Coverage by Tier (Where It Matters Most)

Strategic accounts *must* have phones. This surfaces the critical gaps.

```
SELECT
  Support_Tier__c,
  COUNT(Id)                TotalAccounts,
  COUNT(Phone)             WithPhone,
  COUNT(Id) - COUNT(Phone) MissingPhone
FROM Account
```

```
WHERE IsDeleted = FALSE
GROUP BY Support_Tier__c
ORDER BY Support_Tier__c
```

Note: Only runs after `Support_Tier__c` formula is deployed (A.1).

B.3 — Phone Format Consistency

Salesforce does not enforce phone format. Variations cause lookup failures.

```
SELECT Phone, COUNT(Id) AccountCount
FROM Account
WHERE Phone != NULL
      AND IsDeleted = FALSE
GROUP BY Phone
HAVING COUNT(Id) > 1
ORDER BY COUNT(Id) DESC
LIMIT 50
```

Looks for duplicate phones across Accounts (data quality issue — same phone on multiple accounts breaks tier lookup).

B.4 — Phone Format Patterns (Classify Current State)

Run via Apex Anonymous (SOQL alone can't easily do regex classification). This is a **diagnostic script, not a deployable artifact**:

```
Map<String,Integer> patternCounts = new Map<String,Integer>();
List<String> patterns = new List<String>{
    '^\\+91[0-9]{10}$',      // +91XXXXXXXXXX
    '^91[0-9]{10}$',        // 91XXXXXXXXXX
    '^[0-9]{10}$',          // 10-digit
    '^0[0-9]{10,11}$',      // leading 0
    '^+[0-9]+$'             // any international
    '.*[-\\s()]*'           // contains separators (space, dash, parens)
};

for (String p : patterns) patternCounts.put(p, 0);
patternCounts.put('OTHER', 0);
patternCounts.put('NULL', 0);

for (Account a : [SELECT Phone FROM Account WHERE IsDeleted = FALSE LIMIT 50000]) {
    if (a.Phone == null) { patternCounts.put('NULL', patternCounts.get('NULL') + 1);
    continue; }
    Boolean matched = false;
    for (String p : patterns) {
        if (Pattern.matches(p, a.Phone)) {
            patternCounts.put(p, patternCounts.get(p) + 1);
            matched = true;
            break;
        }
    }
}
```

```

    }
    if (!matched) patternCounts.put('OTHER', patternCounts.get('OTHER') + 1);
}

for (String p : patternCounts.keySet()) {
    System.debug(p + ' : ' + patternCounts.get(p));
}

```

What to look for: If > 20% land in `OTHER` or `contains separators`, normalization is needed before phone-based lookup will work.

B.5 — Contact Phone Coverage (Fallback Lookup)

When `Account.Phone` is blank, `Contact.Phone` on primary contacts is a reasonable fallback.

```

SELECT
    Account.Support_Tier__c      Tier,
    COUNT(DISTINCT AccountId)    AccountsWithContacts,
    COUNT(Id)                    TotalContacts,
    COUNT(Phone)                 ContactsWithPhone,
    COUNT(MobilePhone)           ContactsWithMobile
FROM Contact
WHERE AccountId != NULL
      AND Account.IsDeleted = FALSE
GROUP BY Account.Support_Tier__c
ORDER BY Account.Support_Tier__c

```

B.6 — Accounts With No Phone on Account OR Contact

The worst-case: a Strategic account where neither the Account record nor any Contact has a reachable phone number. **IVR callout will fail for these customers.**

```

SELECT Id, Name, Support_Tier__c, Account_CSM__r.Name
FROM Account
WHERE Support_Tier__c IN ('STRATEGIC', 'HYPERCARE')
      AND Phone = NULL
      AND Id NOT IN (
        SELECT AccountId FROM Contact
        WHERE AccountId != NULL
              AND (Phone != NULL OR MobilePhone != NULL)
      )
      AND IsDeleted = FALSE
ORDER BY Name

```

Expected action: The output list goes to Account owners / CSMs for immediate phone-number enrichment.

B.7 — Duplicate Phone Numbers Across Accounts (Conflict Risk)

If two different Accounts share a phone number, IVR lookup will return the first match — which may be wrong.

```

SELECT Phone, COUNT(Id) AccountCount,
       MAX(Name) SampleName1
FROM Account
WHERE Phone != NULL
      AND IsDeleted = FALSE
GROUP BY Phone
HAVING COUNT(Id) > 1
ORDER BY COUNT(Id) DESC
LIMIT 100

```

Expected action: Manual review. Typically resolved by:

- Keeping phone on the Setup/Tenant account, removing from parent
- Using `Customer_Universal_Account__c` (from `accounts_revenue__c`) as the canonical account
- Introducing a dedicated `Primary_Support_Phone__c` field on Account that Data team curates

B.8 — Phone Normalization Helper (Apex Anonymous)

Once the audit is complete, normalize phone numbers to a single format. **Run in sandbox first. DO NOT run in production without a backup.**

```

// Normalize all Account.Phone values to +91XXXXXXXXXX format for 10-digit numbers
// DRY RUN (remove the return; to actually update)
List<Account> toUpdate = new List<Account>();

for (Account a : [SELECT Id, Phone FROM Account WHERE Phone != NULL AND IsDeleted =
FALSE LIMIT 10000]) {
    String original = a.Phone;
    String normalized = original.replaceAll('[\\s\\-\\(\\)\\+]', ''); // strip
whitespace, dashes, parens, plus

    // If starts with 91 and is 12 digits, keep as +91XXXXXXXXXX
    if (normalized.length() == 12 && normalized.startsWith('91')) {
        normalized = '+' + normalized;
    }
    // If 10 digits, assume Indian mobile, prefix +91
    else if (normalized.length() == 10 && Pattern.matches('^([6-9][0-9]{9})$',
normalized)) {
        normalized = '+91' + normalized;
    }
    // If starts with 0 and is 11 digits (leading zero Indian mobile), strip zero,
prefix +91
    else if (normalized.length() == 11 && normalized.startsWith('0')) {
        normalized = '+91' + normalized.substring(1);
    }
    // Otherwise leave alone (international numbers, unusual formats)
    else {
        continue;
    }

    if (normalized != original) {

```

```
        a.Phone = normalized;
        toUpdate.add(a);
    }
}

System.debug('Would update ' + toUpdate.size() + ' Accounts');
// DRY RUN – uncomment to actually update:
// update toUpdate;
```

Rollback plan: Run a SELECT + CSV export of Id, Phone BEFORE executing the update. Keep the CSV as a rollback artifact for 90 days.

Part C — Pre-Deploy Validation Checklist

Run these before deploying the Flow in Case Flow Specification [02](#).

- ☐ B.1: Baseline phone coverage > 60%
 - ☐ B.2: Strategic + Hypercare phone coverage = 100% (anything less is a blocker)
 - ☐ B.6: Zero accounts returned (or all resolved via Data team enrichment)
 - ☐ B.7: No conflicting duplicate phones on Strategic/Hypercare accounts
 - ☐ A.1 Support_Tier__c deployed and returning expected values on spot-checked accounts
 - ☐ A.3–A.6 Case fields deployed and visible on Case page layout
-

Part D — Ongoing Monitoring

After deployment, track phone data quality monthly:

```
-- Strategic/Hypercare accounts that lost their phone in the last month
SELECT Id, Name, Support_Tier__c, LastModifiedDate
FROM Account
WHERE Support_Tier__c IN ('STRATEGIC', 'HYPERCARE')
    AND Phone = NULL
    AND LastModifiedDate = LAST_N_DAYS:30
ORDER BY LastModifiedDate DESC
```

Set up a monthly report + alert to CSM leadership on any Strategic account with missing phone data.